

Git Workflow Guide for Student Contributions

This document explains, **step by step**, how you should work with the project GitHub repository. Follow these instructions **exactly**. The goal is simple:

- You **never** push code directly to the main branch
- You work **only in your own branch**
- Your work remains isolated, clean, and reviewable

Read slowly and execute each step carefully.

1. Clone the Repository (One-Time Setup)

Cloning means creating a local copy of the remote repository on your system.

Command

```
git clone <REPOSITORY_URL>
```

Explanation

- `git clone` downloads the full repository history
- `<REPOSITORY_URL>` is the GitHub HTTPS link of the project
- This creates a new folder with the project name

After cloning, move into the project directory:

```
cd <project-folder-name>
```

2. Verify Repository Status

Before doing anything else, confirm that Git is properly set up.

Command

```
git status
```

Explanation

- Shows your current branch
 - Shows whether there are uncommitted changes
 - Right after cloning, it should say:
 - On branch main
 - working tree clean
-

3. Always Pull the Latest Main Branch

Before creating your own branch, make sure your local main branch is up to date.

Commands

```
git checkout main  
git pull origin main
```

Explanation

- git checkout main switches to the main branch
 - git pull origin main syncs your local main with GitHub
 - This avoids future merge conflicts
-

4. Create Your Own Branch (Mandatory)

You **must** create a personal branch before starting any work.

Naming Convention

Use your name or roll number:

```
feature-<your-name>
```

Command

```
git checkout -b <your-branch-name>
```

Explanation

- -b creates a new branch and switches to it

- Your work will now be isolated from main
- You should never code on `main`

Verify branch:

```
git branch
```

The current branch will have a `*` next to it.

5. Add Your Work (Code / Files)

Now you can start implementing your assigned task.

- Create new files
- Modify existing files (only if required)
- Follow project folder structure strictly

Check changes:

```
git status
```

6. Stage Your Changes

Staging means selecting files that will go into the next commit.

To stage specific files

```
git add <file-name>
```

To stage everything

```
git add .
```

Explanation

- Files move from **working directory** → **staging area**
 - Nothing is saved in Git history yet
-

7. Commit Your Changes

A commit is a snapshot of your work.

Command

```
git commit -m "Clear and meaningful commit message"
```

Guidelines for Commit Messages

Good:

- "Add PDF extraction logic"
- "Implement paper retrieval module"

Bad:

- "changes"
 - "done"
 - "updated"
-

8. Push Your Branch to GitHub

Your branch exists locally until you push it.

Command

```
git push origin <your-branch-name>
```

Explanation

- Uploads your branch to GitHub
 - Does **not** affect main
 - Your work is now visible for review
-

9. Keep Your Branch Updated (Important)

If main changes while you are working, update your branch safely.

Commands

```
git checkout main
git pull origin main
git checkout <your-branch-name>
git merge main
```

Explanation

- Brings latest main changes into your branch
- Prevents conflicts during final merge

If conflicts appear:

- Do not panic
- Read conflict markers carefully
- Resolve manually

10. What You Must NOT Do

❌ Do NOT push to main ❌ Do NOT work without a branch ❌ Do NOT commit broken or incomplete code
❌ Do NOT rename project structure arbitrarily

11. How Your Work Will Be Evaluated

You should be ready to explain:

- What your branch contains
- What files you modified
- Why you structured the solution that way
- How your code fits into the overall system

I will check your:

- Branch discipline
- Commit clarity
- Code correctness
- Understanding of Git workflow

Follow this process strictly. This workflow is **industry-standard** and non-negotiable.